

几何计算前沿大作业

陈彦涛 2400012972

Jun 2026

1 选题背景

计图挑战赛的点云去噪任务。对于一个 50000 个点的点云，在归一化到单位球后存在标准差为 0.005-0.020 的正态分布误差，需要逐点生成位移进行回归。网址：第六届计图人工智能挑战赛

评价标准为

1. 对回归后的点云的每个点寻找原始点云中离它最近的点的距离。
2. 回归后的点云距离原始点云对应点的距离。

两个方向都以预测 0 位移作为 0 分，预测完全准确（距离为 0）作为 50 分。两个分数求和。目前场上最高分约 83 分。

2 模型结构

2.1 模型主体

使用 octformer 作为模型主体，使用计图框架的 jsprasetensor 替代原本实现中的八叉树实现稀疏卷积。经过测试，这个 tensor 使用 stride=2 的卷积进行下采样和上采样的过程都不会打乱原有的顺序，并且 Conv3dTranspose 的上采样能够利用哈希复原下采样之前的点，因此尽管没有显式的八叉树结构，jsprasetensor 仍然非常适配 octformer 使用莫顿码排序的思路。

依据在简单数据集上的实验结果和经验，设置体素大小为 0.015。由于每次输入的是 1000 个点的局部点云，因此只用了 3 层 transformer，通道数为 [96, 192, 192]，transformer 块数分别为 [4, 4, 2]，头数为 [6, 6, 6]。

2.2 初始特征输入

由于 jsprasetensor 要求进行体素化，因此如何将点转换为稀疏体素的特征是十分重要的。

一开始的实现使用原本的 PatchEmbd 代码，以 voxel_size/4 为初始体素大小，体素内将点与体素中心的相对坐标作为初始特征。然后通过两次 Stride=2 的卷积进行下采样，得到初始的体素编码。然而在实际测试中，这个做法会在几个 epoch 以内就使得所有的预测都收敛到 0 向量的局部最优解，说明这种依赖卷积的编码方案缺乏对周边环境足够细致、足够鲁棒的感知能力。

为了检查是代码哪一部分出现了问题，首先更换成了三层动态建图的 EdgeConv，随后依次测试了一层 EdgeConv，对体素内的点进行 PointNet 风格的 NLP+MaxPooling、对距离体素最近的 k 个点执行 NLP+MaxPooling。

在简单测试任务上的实验结果表明，这几个编码方式都能够摆脱 0 预测，并且实际上它们的效果并没有太大的差别。最终选择的方案是对每一个激活体素，选择离体素中心最近的 16 个点，将其与体素中心的相对坐标作为初始输入通过一个 NLP，然后取 Max，得到每个体素的初始特征。

2.3 解码器

从低分辨率到高分辨率的下采样过程是简单的，只需要进行逆卷积即可。最终能够得到每一层体素的特征。

为了从体素特征得到每个点的预测值，有三线性插值、周围 8 个体素分别预测偏移取平均、最近的 k 个体素分别预测偏移取平均几种方案。由于体素并不是很密集，周围 8 个体素平均有 40%60% 的概率为空，所以经验性地舍弃了三线性插值的做法。

而剩下两种做法实际上可以相互转化。经过实验，取最近的 k 个体素能够有效提升训练质量。而在预测时适当降低用来预测的体素个数可以避免远距离预测，提升预测质量。

最终选择的方案是：每个点预测时，获取最近的 k 个体素的特征，分别和跟相对位移拼接并通过一个 NLP 得到 3 维的向量。将 k 个预测向量取平均值。

2.4 预测

由于设备内存限制，因此对点云进行最远点采样，得到多个重叠但是合并起来能够完整覆盖原始点云的局部点云。每次对局部点云进行预测，每个点选择距离最近的一个局部点云的预测作为最终的预测。

为了提升运行效率，同时也是为了更好保留原始特征，只对初始的 `pc_noisy` 进行编码，随后根据编码进行多步去噪。这也是为什么要选择最近的 k 个体素解码而不是周围 8 个相邻体素的原因。

3 实验

3.1 数据集

使用挑战赛提供的训练集，对模型采样 32768 个点之后对点云进行归一化和旋转，随后在点云的局部采样 1000 个点得到的局部点云，在该点云上增加独立同分布的噪声得到进行训练，每个 batch 含 16 个点云，每个 epoch 含 625 个 batch。

测试集为挑战赛的测试集。

3.2 损失函数

根据预测时的方案，训练时使用 `pc_noisy` 进行编码，但是使用 `pc_noisy` 与 `pc_clean` 线性加权得到的 `pc_mix` 进行预测，以匹配预测时的情况。

为了防止 k 个体素的预测相互影响，因此选择损失函数

$$loss = \frac{1}{nk} \sum_{i=0}^n \sum_{j=0}^k |pred(T_j, X_j - x_i^{mix}) - goal_i|^2$$

即要求每一对预测都接近目标值，而不是相互掣肘，让平均值接近目标。

开始时的目标设计为 $goal_i = x_i^{clean} - x_i^{noisy}$ ，训练后得分为 71。

然而实际上预测值应当去往模型上离噪声点最近的点而不是原始干净点，因为干净点相对于模型上离噪声点最近的点的方向实际上是不可区分的，在正态分布的噪声下，每一个方向都有可能。因此直接预测干净点不仅会降低收敛速度，还会增加过拟合的风险。调整 $goal_i$ 为 $x_i^{closest} - x_i^{noisy}$ 之

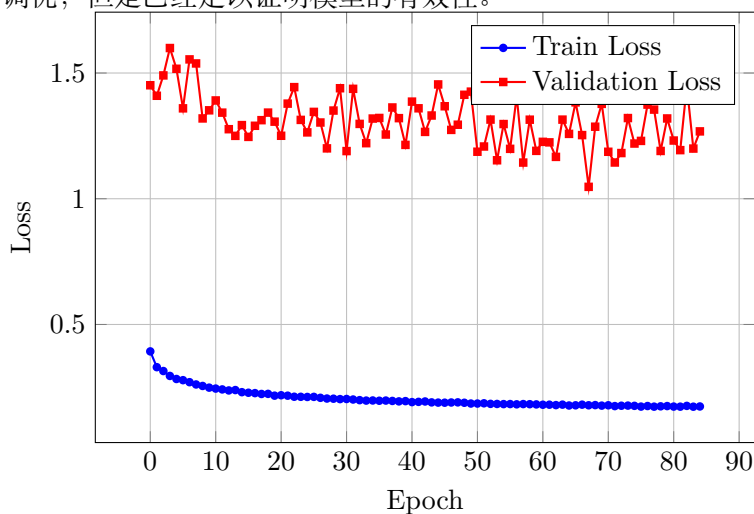
后收敛明显更加稳定，得分提升到了 75 分。其中距离原干净点损失项从 50 提升到了 65，说明了策略的有效性。

3.3 实验设置

使用 Adam 优化器，学习率为 $5e-6$ ，训练 80 个 epoch。

3.4 训练结果

最终取得了 75 分的成绩。由于时间和条件所限，没有更进一步对模型进行调优，但是已经足以证明模型的有效性。



训练损失和测试损失。其中训练损失是预测模型上的最近点，测试损失是预测模型上的原始点，因此测试损失会偏大。

4 代码运行方法

库依赖: jittor, jsparse, igl, scipy

训练和预测方法:

```
python run.py --task configs/task/train_octformer
python run.py --task configs/task/predict_octformer
```

需要先从挑战赛官网下载数据集。

模型文件: experiments/vm/checkpoint_20.pkl。由于文件太大所以放在了网盘。

模型代码:src/model/mymodel.py,src/model/myheader.py,src/model/octformer.py,src/model/encode.py, src/model/decode.py,